

M03 — Divide & Conquer and Recurrences

Sources: CLRS Ch. 4 (Divide-and-Conquer: matrix multiplication, Strassen, substitution, recursion trees, master method, Akra–Bazzi) · Skiena Ch. 5 (Divide and Conquer: binary search, recurrences, Karatsuba, largest subrange, closest pair, parallelism, convolution)

Big Idea

Divide and conquer is three steps — **Divide** into smaller instances of the same problem, **Conquer** them recursively, **Combine** the results — plus a base case. Its power is that the analysis is mechanical: the three steps translate directly into a recurrence $T(n) = aT(n/b) + f(n)$, and the recurrence is solved by comparing the **driving function** $f(n)$ against the **watershed function** $n^{\log_b a}$. Whichever is bigger wins: if the watershed wins, the cost lives in the leaves ($\Theta(n^{\log_b a})$); if they tie, every level costs the same and you pay a $\log n$ factor; if the driving function wins, the root dominates ($\Theta(f(n))$). That comparison is the master theorem, and once you see the recursion tree behind it, the three cases stop being something to memorize. The strategic insight — the one that produced Strassen and Karatsuba — is that **reducing a (the branching factor) beats reducing $f(n)$** : trading one recursive multiplication for a handful of extra additions changes the exponent, and nothing else in the recurrence can do that. Remember: *bushiness of the recursion tree decides the exponent*.

What You Should Be Able To Do After This Chapter

- Turn any recursive algorithm into its recurrence, correctly identifying a , b , and $f(n)$.
- Apply the master theorem's three cases, including checking the **polynomial separation** and the **regularity condition**, and recognize the two gaps where it does not apply.
- Solve a recurrence by **substitution**, including the "subtract a lower-order term" trick, and avoid the two classic fallacies (asymptotic notation in the hypothesis; proving $O(n)$ instead of $\leq cn$).
- Draw a **recursion tree**, compute per-level costs and the number of leaves, and read off whether the root, the levels, or the leaves dominate.

- Explain why $T(n) = 8T(n/2) + \Theta(1)$ is $\Theta(n^3)$ while $T(n) = 2T(n/2) + \Theta(n)$ is $\Theta(n \log n)$ — in terms of bushiness.
 - Derive Karatsuba and Strassen from the "trade a multiplication for additions" idea, and state their recurrences and exponents.
 - Write correct binary search (and its lower-bound/upper-bound variants) in C++, first try, with the overflow-safe midpoint.
 - Solve the largest-subrange and closest-pair problems by the "left, right, or straddling the middle" template.
 - Use Akra–Bazzi when subproblem sizes differ, and know the polynomial-growth condition that lets you ignore floors and ceilings.
-

1. The method, and what a recurrence is

Unified Understanding

The three steps [CLRS Ch. 4, p.76]:

- **Divide** the problem into one or more subproblems that are smaller instances of the same problem.
- **Conquer** the subproblems by solving them recursively.
- **Combine** the subproblem solutions to form a solution to the original problem.
- **Base case:** if the problem is small enough, solve it directly without recursing.

Skiena's framing of *when it pays*:

Whenever the merging takes less time than solving the two subproblems, we get an efficient algorithm. [Skiena Ch. 5, p.147]

And his contrast with the other decomposition paradigm:

***Dynamic programming** typically removes one element from the problem, solves the smaller problem, and then adds back the element to the solution of this smaller problem in the proper way. **Divide and conquer** instead splits the problem into (say) halves, solves each half, then stitches the pieces back together.*

Skiena's honest caveat, which is worth taking seriously:

*Beyond binary search and its many variants, I find it to be a **difficult design technique to apply in practice**.*

That is why this module spends more time on *analyzing* divide and conquer than on inventing it — and why the interview-relevant skill here is recurrence-solving, not D&C invention.

Recurrences [CLRS Ch. 4, p.76]

*A **recurrence** is an equation that describes a function in terms of its value on other, typically smaller, arguments.*

Formally: an equation or inequality describing a function over the integers or reals **using the function itself**, containing two or more cases. A case that invokes the function recursively is a **recursive case**; one that doesn't is a **base case**. The recurrence is **well defined** if at least one function satisfies it.

Skiena's point that recurrences are a general notation, not a D&C-only tool [§5.3, p.152]:

$a_n = a_{n-1} + 1,$	$a_1 = 1 \rightarrow$	$a_n = n$	(linear)
$a_n = 2a_{n-1},$	$a_1 = 1 \rightarrow$	$a_n = 2^{n-1}$	(exponential)
$a_n = n \cdot a_{n-1},$	$a_1 = 1 \rightarrow$	$a_n = n!$	(factorial)
$F_n = F_{n-1} + F_{n-2}$			(Fibonacci)

Recurrence relations provide a way to analyze recursive structures, such as algorithms.

Algorithmic recurrences — CLRS's convention, and why it's safe

A recurrence $T(n)$ is **algorithmic** if, for every sufficiently large threshold constant $n_0 > 0$:

1. For all $n < n_0$, $T(n) = \Theta(1)$.
2. For all $n \geq n_0$, every recursion path terminates in a defined base case within finitely many invocations.

Whenever a recurrence is stated without an explicit base case, we assume that the recurrence is algorithmic.

Why this is legitimate: property 1 holds because there are finitely many inputs below n_0 , and calling and returning from a procedure costs at least some positive constant. Property 2 must hold or **the algorithm isn't correct** — it would loop forever. So any correct D&C algorithm's worst-case recurrence is automatically algorithmic. [CLRS p.77]

Three conventions that follow, all of which you should adopt:

CONVENTION	JUSTIFICATION
Omit the base case	It is always $\Theta(1)$; the asymptotic solution doesn't depend on the threshold.
Drop floors and ceilings	$T(n) = 2T(n/2) + \Theta(n)$ instead of $T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n)$. §4.7 gives sufficient conditions (below).
Inequality $\rightarrow$ one-sided bound	$T(n) \leq 2T(n/2) + \Theta(n)$ gives an O -solution; $\geq$ gives an Ω -solution.

Subproblem-size taxonomy

RECURRENCE	SHAPE	SOLUTION
$T(n) = 2T(n/2) + \Theta(n)$	equal halves, linear combine	$\Theta(n \log n)$ — merge sort
$T(n) = 8T(n/2) + \Theta(1)$	equal halves, 8-way, $O(1)$ combine	$\Theta(n^3)$ — naive recursive matrix multiply
$T(n) = 7T(n/2) + \Theta(n^2)$	equal halves, 7-way	$\Theta(n^{\lg 7}) = O(n^{2.81})$ — Strassen
$T(n) = T(n/3) + T(2n/3) + \Theta(n)$	unequal split	$\Theta(n \log n)$
$T(n) = T(n/5) + T(7n/10) + \Theta(n)$	unequal, shrinking total	$\Theta(n)$ — median-of-medians selection (M05)
$T(n) = T(n-1) + \Theta(1)$	constant shrinkage	$\Theta(n)$ — recursive linear search

*The vast majority of efficient divide-and-conquer algorithms solve subproblems that are **a constant fraction** of the size of the original. [CLRS p.79]*

2. Algorithm: Binary Search — "the mother of all divide-and-conquer algorithms"

Problem

Find key q in a **sorted** array $S[0..n-1]$; report its index or that it is absent.

Core intuition

Compare q to the middle element. One comparison eliminates **half** the remaining candidates. Repeat.

Mental model

Twenty questions. Skiena's framing [§5.1, p.148]: a printed dictionary holds 50,000–200,000 words, so $\lg(200000) \approx 18$ — the second player in twenty questions always wins by binary search.

Pseudocode

```
BinarySearch(S, q, lo, hi)
    if lo > hi: return NOT_FOUND
    mid = (lo + hi) / 2
    if S[mid] == q: return mid
    if S[mid] > q: return BinarySearch(S, q, lo, mid - 1)
    else: return BinarySearch(S, q, mid + 1, hi)
```

→ **C++ implementation:** [A1 BinarySearch](#)

Why it works — loop invariant

If q is present in the original array, it lies within $S[lo..hi]$.

Initialization: $lo = 0, hi = n-1$ — the whole array. **Maintenance:** the array is sorted, so $S[mid] > q$ implies q cannot be at mid or to its right; symmetrically for $<$.

Termination: either $S[mid] == q$ (found), or $lo > hi$ — an empty range, so by the invariant q was never present. ■

Complexity

$$T(n) = T(n/2) + \theta(1) \rightarrow \theta(\log n)$$

Master theorem case 2: $a = 1$, $b = 2$, watershed $n^{\log_2 1} = n^0 = 1$, driving $f(n) = \Theta(1) = \Theta(n^0 \lg^0 n)$, so $T(n) = \Theta(\lg n)$. [CLRS Ex. 4.5-3]

Space: $\Theta(1)$ iterative; $\Theta(\lg n)$ stack if recursive.

C++ Implementation

```
#include <vector>

// Classic binary search. Returns index of key, or -1 if absent.
template <typename T>
int binarySearch(const vector<T>& v, const T& key) {
    int lo = 0, hi = static_cast<int>(v.size()) - 1;
    while (lo <= hi) {
        const int mid = lo + (hi - lo) / 2;    // overflow-safe
        if (v[mid] == key)    return mid;
        else if (v[mid] < key) lo = mid + 1;
        else                  hi = mid - 1;
    }
    return -1;
}

// First index i with v[i] >= key (== lower_bound). Returns
// v.size() if none.
template <typename T>
int lowerBound(const vector<T>& v, const T& key) {
    int lo = 0, hi = static_cast<int>(v.size());    // NOTE: half-
    open [lo, hi)
    while (lo < hi) {
        const int mid = lo + (hi - lo) / 2;
        if (v[mid] < key) lo = mid + 1;
        else              hi = mid;
    }
    return lo;
}

// First index i with v[i] > key (== upper_bound).
template <typename T>
int upperBound(const vector<T>& v, const T& key) {
    int lo = 0, hi = static_cast<int>(v.size());
    while (lo < hi) {
```

```

        const int mid = lo + (hi - lo) / 2;
        if (!(key < v[mid])) lo = mid + 1;           // v[mid] <= key
        else
            hi = mid;
    }
    return lo;
}

// Number of occurrences of key in a sorted vector, in O(log n)
// regardless of how many copies there are.
template <typename T>
int countOccurrences(const vector<T>& v, const T& key) {
    return upperBound(v, key) - lowerBound(v, key);
}

```

Implementation notes

- `lo + (hi - lo) / 2`, never `(lo + hi) / 2`. The latter overflows `int` for arrays over $\sim 2^{30}$ elements. This exact bug shipped in `java.util.Arrays.binarySearch` and in the JDK's merge sort for nine years.
- **Half-open** `[lo, hi)` for the bound variants, **closed** `[lo, hi]` for the classic **search**. Mixing the two conventions is the single biggest source of binary-search bugs. Pick one per function and never deviate.
- `lowerBound/upperBound` **always terminate** because `hi - lo` strictly decreases: when `lo = mid + 1`, `mid ≥ lo` so `lo` grows; when `hi = mid`, `mid < hi` so `hi` shrinks.
- `upperBound` uses `!(key < v[mid])` so only `operator<` is required — matching STL conventions.
- In practice use `std::lower_bound` / `std::upper_bound` / `std::equal_range`. Write them by hand only in interviews or when the predicate isn't a simple comparison.

Common bugs

- `(lo + hi) / 2` overflow.
- Mixing `[lo, hi]` and `[lo, hi)` inside one function → infinite loop or off-by-one.
- `while (lo < hi)` with `hi = mid - 1` → skips the answer.
- `while (lo <= hi)` with `hi = mid` → infinite loop when `lo == hi`.

- Assuming the array is sorted when it isn't. Binary search on unsorted data silently returns garbage.

Three variants worth knowing [Skiena §5.1]

1. Counting occurrences in $O(\lg n)$. Naively you find one occurrence in $O(\lg n)$ then scan outward — $O(\lg n + s)$, which degrades to $\Theta(n)$ when the whole array is one key. Instead, **delete the equality test** so every search "fails", and it terminates at a block boundary. Reverse the comparison direction to find the other boundary. Two $O(\lg n)$ searches, done. (Equivalently: `upperBound - lowerBound`, as above.)

Skiena's other trick for the same problem: search for $k - \epsilon$ and $k + \epsilon$ with a modified routine that returns the insertion position — both searches fail with no intervening keys, and their positions bracket the block.

2. One-sided (galloping / exponential) binary search. Array of 0s followed by an unbounded run of 1s; find the transition, with **no known bound n** . Probe `A[1]`, `A[2]`, `A[4]`, `A[8]`, `A[16]`, ... until you hit a 1, then binary search inside that window. Finds the transition point `p` in at most $2\lceil \lg p \rceil$ comparisons, regardless of the array's actual size.

One-sided binary search is useful whenever we are looking for a key that lies close to our current position.

```
// Finds the smallest index p >= 1 with pred(p) true, given pred is
// monotone
// (false...false true...true) and unbounded. O(log p) evaluations
// of pred.
template <typename Pred>
long long exponentialSearch(Pred pred) {
    long long hi = 1;
    while (!pred(hi)) hi *= 2;           // doubling phase: <=
lg p probes
    long long lo = hi / 2 + 1;          // pred(hi/2) was false
    while (lo < hi) {                   // binary phase: <= lg
p probes
        const long long mid = lo + (hi - lo) / 2;
        if (pred(mid)) hi = mid;
        else lo = mid + 1;
    }
    return lo;
}
```


3. Bisection for roots (square roots, and any continuous f). $\sqrt{n}$ lies in $[1, n]$. Test $m = (l+r)/2$: if $n \geq m^2$ recurse right, else left. After $\lceil \lg n \rceil$ rounds you have $\sqrt{n}$ to $\pm 1/2$. Generalizes: if f is continuous with $f(l) > 0$ and $f(r) < 0$, a root lies between them, and the sign of $f(m)$ halves the window.

*Root-finding algorithms converging faster than binary search are known... Still, binary search is **simple, robust, and works as well as possible without additional information** on the nature of the function.*

Recognition pattern for all of these: you have a **monotone predicate** over an ordered domain, and you want the boundary. That is the general form — "binary search on the answer" — and it is one of the highest-frequency competitive-programming patterns. See the Recognition Table.

3. War Story: Finding the Bug in the Bug — parallelizing binary search

[Skiena §5.2, p.150]

Skiena's team built synthetic attenuated viruses by replacing a 1,200-base gene region — and killed the virus. Somewhere in those 1,200 bases was a hidden survival signal. Finding it by binary search would take $\lg(16) = 4$ sequential rounds of experiment, and **each round took Yutong a month** to synthesize, clone and grow.

*Yutong realized that the power of binary search came from **interaction**: the query we make in round r depends upon the answers to queries in rounds 1 through $r-1$.*

***Binary search is an inherently sequential algorithm.** When each individual comparison is a slow and laborious process, suddenly $\lg n$ comparisons doesn't look so good.*

The fix. Run all four "rounds" *simultaneously* as four separate designs, each half red (dead-strain sequence) and half green (viable), arranged so that the 16 candidate regions get **16 distinct red/green column patterns**. The live/dead outcome of the four designs is a 4-bit code naming the region.

This is exactly the difference between **adaptive** and **non-adaptive** search. Adaptive binary search takes $\lg n$ sequential steps; non-adaptive requires $\lg n$ *parallel* queries, but each query may be an arbitrary subset rather than a contiguous half. Skiena's designs are the rows of the binary-encoding matrix — the same construction as a Hamming-style identification code.

The engineering lesson generalizes far beyond biology. Whenever each probe is expensive in *latency* but cheap in *throughput* — a batch of CI runs, a set of A/B experiments, a round of network probes — convert the adaptive binary search into a one-round set of $\lg n$ subset queries.

4. Method 1: Substitution

The method [CLRS §4.3, p.90]

1. **Guess** the form of the solution, using symbolic constants.
2. **Use mathematical induction** to show it works, and solve for the constants.

It's usually best not to try to do both at the same time. Rather than trying to prove a Θ -bound directly, first prove an O -bound, and then prove an Ω -bound.

Worked example — $T(n) = 2T(\lfloor n/2 \rfloor) + \Theta(n)$ is $O(n \lg n)$

Hypothesis: $T(n) \leq cn \lg n$ for all $n \geq n_0$, constants chosen later.

$$\begin{aligned} T(n) &\leq 2(c\lfloor n/2 \rfloor \lg \lfloor n/2 \rfloor) + \Theta(n) \\ &\leq 2(c(n/2) \lg(n/2)) + \Theta(n) \\ &= cn \lg(n/2) + \Theta(n) \\ &= cn \lg n - cn \lg 2 + \Theta(n) \\ &= cn \lg n - cn + \Theta(n) \\ &\leq cn \lg n \end{aligned}$$

— the last step holding if **c and n_0 are large enough that cn dominates the anonymous function hidden by $\Theta(n)$** .

Base cases. Need $T(n) \leq cn \lg n$ for $n_0 \leq n < 2n_0$. Requires $n_0 > 1$ so that $n \lg n > 0$; pick $n_0 = 2$. Since the recurrence is algorithmic, $T(2)$ and $T(3)$ are constants. Pick $c = \max\{T(2), T(3)\}$: then $T(2) \leq c < (2 \lg 2)c$ and $T(3) \leq c < (3 \lg 3)c$. ■

In the algorithms literature, people rarely carry out their substitution proofs to this level of detail, especially in their treatment of base cases. ... You ground the induction on a range of values from a convenient positive constant n_0 up to some constant $n_0' > n_0$ such that for $n \geq n_0'$, the recurrence always bottoms out in a constant-sized base case. [CLRS p.91]

Translation: in an interview, do the inductive step carefully and wave at the base cases. Do not skip the inductive step.

Making a good guess

There is no general method. Three heuristics [CLRS §4.3, p.92]:

1. Pattern-match against recurrences you know. $T(n) = 2T(n/2 + 17) + \theta(n)$ looks like merge sort; the $+17$ shouldn't matter for large n since both still cut n nearly in half. Guess $O(n \lg n)$ and verify.

2. Squeeze from both sides. Start with a loose $\Omega(n)$ (from the $\theta(n)$ term) and a loose $O(n^2)$, then alternate lowering the upper bound and raising the lower until they meet.

3. Draw the recursion tree (§5 below) and read the guess off it.

The trick of the trade: **subtract a lower-order term**

Consider $T(n) = 2T(n/2) + \theta(1)$. Guess $T(n) \leq cn$:

$$T(n) \leq 2(c(n/2)) + \theta(1) = cn + \theta(1) \quad \times \text{ does NOT give } \leq cn$$

The guess $O(n)$ is **correct and tight** — the problem is the hypothesis is too weak.

Strengthen it to $T(n) \leq cn - d$ for a constant $d \geq 0$:

$$\begin{aligned} T(n) &\leq 2(c(n/2) - d) + \theta(1) \\ &= cn - 2d + \theta(1) \\ &\leq cn - d - (d - \theta(1)) \\ &\leq cn - d \end{aligned} \quad \checkmark \text{ as long as } d \text{ exceeds the } \theta\text{'s hidden constant}$$

Why subtract rather than add — this is the part people get backwards:

*When the recurrence contains more than one recursive invocation, if you **add** a lower-order term to the guess, then you end up adding it once **for each** recursive invocation. Doing so takes you even further away from the inductive hypothesis. On the other hand, if you **subtract** a lower-order term, then you get to subtract it once for each of the recursive invocations. [CLRS p.93]*

With coefficient 2 , you subtract d twice and only need to give back d once — the slack is yours.

The two pitfalls

Pitfall 1 — asymptotic notation inside the inductive hypothesis. This "proves" $T(n) = 2T(\lfloor n/2 \rfloor) + \theta(n)$ is $O(n)$:

$$T(n) \leq 2 \cdot O(\lfloor n/2 \rfloor) + \theta(n) = 2 \cdot O(n) + \theta(n) = O(n) \quad \leftarrow \text{WRONG}$$

The fallacy: **the constant hidden by o changes**. Redo it with an explicit constant and it collapses:

$$T(n) \leq 2(c\lfloor n/2 \rfloor) + \theta(n) \leq cn + \theta(n)$$

$cn + \theta(n)$ is indeed $O(n)$, but the hidden constant must **exceed c** — so you cannot conclude $\leq cn$.

*When using the substitution method, you must be careful that the constants hidden by any asymptotic notation are **the same constants throughout the proof**. Consequently, it's best to **avoid asymptotic notation in your inductive hypothesis and to name constants explicitly**.*

Pitfall 2 — proving the goal instead of the hypothesis.

$$T(n) \leq 2(c\lfloor n/2 \rfloor) + \theta(n) \leq cn + \theta(n) = O(n) \quad \leftarrow \text{WRONG}$$

The mistake is the gap between the **goal** ($T(n) = O(n)$) and the **inductive hypothesis** ($T(n) \leq cn$). Induction requires you to prove **the exact statement of the hypothesis** — here, $T(n) \leq cn$, which you have not.

5. Method 2: Recursion Trees

What it is

In a recursion tree, each node represents the cost of a single subproblem somewhere in the set of recursive function invocations. You typically sum the costs within each level to obtain the per-level costs, and then sum all the per-level costs. [CLRS §4.4, p.95]

*A recursion tree is best used to **generate intuition for a good guess**, which you can then verify by the substitution method. If you are meticulous when drawing out a recursion tree and summing the costs, however, you can use it as a **direct proof**.*

Practical advice: be sloppy when guessing, precise when verifying.

The three quantities to extract

QUANTITY	HOW TO GET IT
Depth	subproblem size at depth i is n/b^i ; hits the base case at $i = \log_b n$
Nodes at depth i	a^i
Cost per level i	$a^i \cdot f(n/b^i)$
Number of leaves	$a^{\log_b n} = n^{\log_b a}$ (by the identity $a^{\log_b c} = c^{\log_b a}$)
Leaf cost	$\Theta(n^{\log_b a})$

Then ask: does the **sequence of level costs** grow, stay flat, or shrink geometrically? That is the whole master theorem.

Worked example — $T(n) = 3T(n/4) + \Theta(n^2)$ [CLRS Fig. 4.1]

```

depth 0:                 $cn^2$ 
cost   $cn^2$ 
depth 1:       $c(n/4)^2$     $c(n/4)^2$     $c(n/4)^2$ 
cost   $(3/16)cn^2$ 
depth 2:    9 nodes, each  $c(n/16)^2$ 
cost   $(3/16)^2cn^2$ 
       $\vdots$ 
depth  $\log_4 n$ :  $3^{\log_4 n} = n^{\log_4 3}$  leaves, each  $\Theta(1)$ 
cost   $\Theta(n^{\log_4 3})$ 

```

Summing:

$$\begin{aligned}
 T(n) &= \sum_{i=0}^{\log_4 n} (3/16)^i cn^2 + \theta(n^{\log_4 3}) \\
 &< cn^2 \sum_{i=0}^{\infty} (3/16)^i + \theta(n^{\log_4 3}) \\
 &= cn^2 \cdot 1/(1 - 3/16) + \theta(n^{\log_4 3}) \quad [\text{geometric series, A.7}] \\
 &= (16/13)cn^2 + \theta(n^{0.793}) \\
 &= O(n^2)
 \end{aligned}$$

The coefficients form a decreasing geometric series, so the whole tree costs a constant multiple of the root. The root dominates.

And the bound is **tight**: the first call alone contributes $\theta(n^2)$, so $\Omega(n^2)$ holds too.

Verifying by substitution. Show $T(n) \leq dn^2$:

$$T(n) \leq 3T(n/4) + cn^2 \leq 3d(n/4)^2 + cn^2 = (3/16)dn^2 + cn^2 \leq dn^2$$

whenever $d \geq (16/13)c$. ✓

Note the two constants. c is given to us by the $\theta(n^2)$ — we can't pick it (though any $c' \geq c$ also works). d we choose freely, and its value depending on c is fine, since d is constant when c is.

Worked example — the unbalanced tree $T(n) = T(n/3) + T(2n/3) + \theta(n)$ [CLRS Fig. 4.2]

	cn		total	
cn				
	c(n/3)	c(2n/3)		total
cn				
	c(n/9)	c(2n/9)	c(2n/9)	c(4n/9)
total	cn			
	:	(unbalanced — leaves at different depths)		

Height. The *longest* path runs down the right edge: $n, (2/3)n, (4/9)n, \dots$. It hits the threshold when $(2/3)^h n < n_0$, giving $h = \lfloor \log_{3/2}(n/n_0) \rfloor + 1 = \theta(\lg n)$.

Internal nodes. Each *full* level sums to exactly cn (the two children's arguments sum back to n). At most cn per level $\times \Theta(\lg n)$ levels = $O(n \lg n)$.

Leaves — and the instructive trap. Tempting bound: the tree sits inside a complete binary tree of height $h = \lfloor \log_{3/2} n \rfloor + 1$, which has $2^h \leq 2n^{\log_{3/2} 2} = O(n^{\{1.71\}})$ leaves. That would swamp the $O(n \lg n)$ internal cost — **and it is not tight.**

Do it properly with a separate recurrence for the leaf count:

$$\begin{aligned} L(n) &= 1 && \text{if } n < n_0 \\ L(n) &= L(n/3) + L(2n/3) && \text{if } n \geq n_0 \end{aligned}$$

Substitution with $L(n) \leq dn$: $L(n) \leq dn/3 + 2dn/3 = dn$ ✓ for any $d > 0$, and $d = 1$ handles the base case. So $L(n) = O(n)$, and the total leaf cost is $\Theta(n)$ — asymptotically less than the internal cost.

$$T(n) = O(n \lg n) + \Theta(n) = O(n \lg n) \quad (\text{and } \Omega(n \lg n), \text{ so } \Theta(n \lg n))$$

The cost of the internal nodes dominates the cost of the leaves, not vice versa. ... You may see recurrences for which the cost of leaves dominates the cost of internal nodes, and then you'll be in better shape if you've had some experience analyzing the number of leaves. [CLRS p.100]

The general lesson: bounding an unbalanced recursion tree by the complete binary tree that contains it is almost always far too generous. Write a recurrence for the leaf count instead.

Skiena's picture of the general tree [Fig. 5.2, p.154]

For $T(n) = aT(n/b) + f(n)$: vertex degree a , partition size n/b^i , height $\log_b n$, width (leaf count) $a^{\log_b n} = n^{\log_b a}$.

6. Method 3: The Master Theorem

Theorem 4.1 (Master theorem) [CLRS §4.5, p.102]

Let $a > 0$ and $b > 1$ be constants, $f(n)$ a **driving function** defined and nonnegative on all sufficiently large reals. Define

$$T(n) = a \cdot T(n/b) + f(n)$$

(where $aT(n/b)$ implicitly means $a'T(\lfloor n/b \rfloor) + a''T(\lceil n/b \rceil)$ for $a' + a'' = a$). Call $n^{\log_b a}$ the **watershed function**. Then:

CASE	CONDITION	SOLUTION	RECURSION TREE BEHAVIOUR
1	$\exists \epsilon > 0$ such that $f(n) = O(n^{\log_b a - \epsilon})$	$T(n) = \Theta(n^{\log_b a})$	level costs grow geometrically; leaves dominate
2	$\exists k \geq 0$ such that $f(n) = \Theta(n^{\log_b a} \lg^k n)$	$T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$	every level costs the same; $\Theta(\lg n)$ levels
3	$\exists \epsilon > 0$ such that $f(n) = \Omega(n^{\log_b a + \epsilon})$ and the regularity condition $a \cdot f(n/b) \leq c \cdot f(n)$ holds for some $c < 1$ and all large n	$T(n) = \Theta(f(n))$	level costs shrink geometrically; root dominates

Skiena's version of the same three, in one sentence each [§5.4, p.155]:

- **Case 1: too many leaves.** The leaf count outweighs the internal evaluation cost $\rightarrow O(n^{\log_b a})$.
- **Case 2: equal work per level.** Cost per level $\times$ number of levels $\rightarrow O(n^{\log_b a} \lg n)$.
- **Case 3: too expensive a root.** Internal cost grows so fast the root dominates $\rightarrow O(f(n))$.

Case 1 holds for heap construction and matrix multiplication, while Case 2 holds for mergesort. Case 3 generally arises with clumsier algorithms, where the cost of combining the subproblems dominates everything.

The two technical details people skip

Polynomial separation, in cases 1 and 3.

*Not only must the watershed function grow asymptotically faster than the driving function, it must grow **polynomially** faster — by at least a factor of $\theta(n^\epsilon)$ for some constant $\epsilon > 0$.*

The separation need not be big. For $T(n) = 4T(n/2) + n^{\{1.99\}}$, watershed is n^2 , driving is smaller by a factor $n^{\{0.01\}}$ — case 1 applies with $\epsilon = 0.01$.

The regularity condition, in case 3. $a \cdot f(n/b) \leq c \cdot f(n)$ for some $c < 1$. It is satisfied by essentially every polynomially bounded function you will meet. It fails for functions that grow slowly locally but quickly overall (CLRS Ex. 4.5-5: $f(n) = 2^{\{\lceil \lg n \rceil\}}$).

Case 2 in practice. "The most common situation for case 2 occurs when $k = 0$ " — watershed and driving function have the same growth, giving $T(n) = \theta(n^{\{\log_b a\} \lg n})$.

Worked applications

RECURRENCE	A	B	WATERSHED $n^{\log_b a}$	$f(n)$	CASE	SOLUTION
$T(n) = 9T(n/3) + n$	9	3	n^2	n	1 ($\epsilon \leq 1$)	$\Theta(n^2)$
$T(n) = T(2n/3) + 1$	1	3/2	$n^0 = 1$	1	2 ($k=0$)	$\Theta(\lg n)$
$T(n) = 3T(n/4) + n \lg n$	3	4	$n^{\{0.793\}}$	$n \lg n$	3 ($\epsilon \approx 0.2$; regularity: $3(n/4) \lg(n/4) \leq$ $(3/4)n \lg n$, $c =$ $3/4$)	$\Theta(n \lg n)$
$T(n) = 2T(n/2) + n \lg n$	2	2	n	$n \lg n$	2 ($k=1$)	$\Theta(n \lg^2 n)$
$T(n) = 2T(n/2) + \Theta(n)$ (merge sort)	2	2	n	$\Theta(n)$	2 ($k=0$)	$\Theta(n \lg n)$
$T(n) = 8T(n/2) + \Theta(1)$ (recursive matmul)	8	2	n^3	$\Theta(1)$	1 (any $\epsilon < 3$)	$\Theta(n^3)$
$T(n) = 7T(n/2) + \Theta(n^2)$ (Strassen)	7	2	$n^{\lg 7}$	$\Theta(n^2)$	1 ($\epsilon = 0.8$)	$\Theta(n^{\lg 7})$
$T(n) = T(n/2) + \Theta(1)$ (binary search)	1	2	1	$\Theta(1)$	2 ($k=0$)	$\Theta(\lg n)$
$T(n) = 2T(n/2) + \Theta(\lg n)$ (heap build)	2	2	n	$\lg n$	1	$\Theta(n)$

When the master method does **not** apply

Three failure modes [CLRS p.105]:

1. Incomparable functions. $f(n)$ may exceed $n^{\log_b a}$ for infinitely many n and fall below it for infinitely many others. Rare in practice.

2. The gap between cases 1 and 2. $f(n) = o(n^{\log_b a})$ but *not polynomially* smaller.

3. The gap between cases 2 and 3. $f(n) = \omega(n^{\log_b a})$ by more than polylog but less than polynomial. Also: regularity fails.

The canonical gap example: $T(n) = 2T(n/2) + n/\lg n$. Watershed is n . Driving $n/\lg n = o(n)$ — but only *logarithmically* smaller, not polynomially: since $\lg n = o(n^\epsilon)$ for every $\epsilon > 0$, we get $n/\lg n = \omega(n^{1-\epsilon})$, so no ϵ makes case 1 work. And case 2 fails because it would need $k = -1$, while $k \geq 0$ is required. Answer (via Akra–Bazzi or substitution): $\Theta(n \lg \lg n)$ [CLRS Ex. 4.6-3].

*Although the master theorem doesn't handle this particular recurrence, it does handle the **overwhelming majority** of recurrences that tend to arise in practice.*

7. Method 4: Akra–Bazzi (unequal subproblems)

When you need it

The master theorem requires **all subproblems the same size**. Akra–Bazzi handles

$$T(n) = f(n) + \sum_{i=1}^k a_i \cdot T(n/b_i)$$

with all $a_i > 0$, all $b_i > 1$, f nonnegative. [CLRS §4.7, p.115]

The method

1. Find the **unique real p** solving $\sum_{i=1}^k a_i/b_i^p = 1$. (It always exists: the sum $\rightarrow \infty$ as $p \rightarrow -\infty$, decreases in p , $\rightarrow 0$ as $p \rightarrow \infty$.)

2. Then

$$T(n) = \Theta\left(n^p \cdot \left(1 + \int_1^n f(x)/x^{p+1} dx\right)\right)$$

Worked example — the selection recurrence

$$T(n) = T(n/5) + T(7n/10) + n \text{ (median-of-medians, M05).}$$

Solve $(1/5)^p + (7/10)^p = 1$. Exactly, $p = 0.83978\dots$ — but **you don't need the exact value**: at $p = 0$ the sum is $2 > 1$, at $p = 1$ it is $9/10 < 1$, so $0 < p < 1$. That's enough.

Using $\int x^k dx = x^{k+1}/(k+1)$ for $k \neq -1$, with $k = -p \neq -1$:

$$\begin{aligned}
 T(n) &= \Theta(n^p (1 + \int_1^n x^{-p} dx)) \\
 &= \Theta(n^p (1 + [x^{1-p}/(1-p)]_1^n)) \\
 &= \Theta(n^p (1 + (n^{1-p} - 1)/(1-p))) \\
 &= \Theta(n^p \cdot \Theta(n^{1-p})) \quad \text{since } 1-p \text{ is a positive constant} \\
 &= \Theta(n)
 \end{aligned}$$

Floors and ceilings — the polynomial-growth condition

Dropping floors and ceilings is **not** always safe, because driving functions can be pathological. The sufficient condition [CLRS p.116]:

$f(n)$ satisfies the **polynomial-growth condition** if there exists $\hat{n} > 0$ such that: for every constant $\varphi \geq 1$, there exists $d > 1$ (depending on φ) with $f(n)/d \leq f(\psi n) \leq d \cdot f(n)$ for all $1 \leq \psi \leq \varphi$ and $n \geq \hat{n}$.

CLRS calls this possibly the hardest definition in the book. **To first order it says $f(\Theta(n)) = \Theta(f(n))$.**

SATISFIES IT	DOES NOT
$f(n) = \Theta(n^\alpha \lg^\beta n \lg^\gamma \lg n)$ for constants α, β, γ	exponentials (2^n) and superexponentials
most polynomially bounded functions in this book	some polynomially bounded functions too

Theorem 4.5. When f satisfies the polynomial-growth condition, replacing every $T(n/b_i)$ with $T(\lceil n/b_i \rceil)$ or $T(\lfloor n/b_i \rfloor)$ **does not change the asymptotic solution.**

Even more slack is available: replacing $T(n/b_i)$ with $T(n/b_i + h_i(n))$ where $|h_i(n)| = O(n/\lg^{1+\epsilon} n)$ leaves the solution unaffected.

Thus, the divide step in a divide-and-conquer algorithm can be **moderately coarse** without affecting the solution to its running-time recurrence.

Master vs Akra–Bazzi

	MASTER THEOREM	AKRA–BAZZI
Subproblem sizes	must be equal	may differ
Math needed	comparison of two functions	calculus (an integral)
Floors/ceilings	ignore freely	need polynomial-growth condition
Ease	much simpler	more general

They are both good tools for your algorithmic toolkit.

8. Algorithm: Karatsuba Multiplication

Problem

Multiply two n -digit integers A and B .

Baseline

Long multiplication is $\Theta(n^2)$ digit products — this is the algorithm you learned in school.

The naive divide and conquer, and why it fails

Split each number in half. With $w = 10^{\{n/2\}}$, write $A = a_0 + a_1w$, $B = b_0 + b_1w$:

$$A \times B = (a_0 + a_1w)(b_0 + b_1w) = a_0b_0 + (a_0b_1 + a_1b_0)w + a_1b_1w^2$$

Four half-size products plus $O(n)$ additions and shifts (multiplication by w is just zero-padding, not a real multiply):

$$\begin{aligned} T(n) &= 4T(n/2) + O(n) \quad \rightarrow \quad \text{watershed } n^{\{\log_2 4\}} = n^2 \quad \rightarrow \quad \text{case 1} \\ &\rightarrow \quad \Theta(n^2) \end{aligned}$$

We divided, but we did not conquer. [Skienna §5.5, p.156]

Core intuition — trade a multiplication for additions

The seed idea, from CLRS's discussion of Strassen: to compute $x^2 - y^2$ you *seem* to need two multiplications, but $x^2 - y^2 = (x+y)(x-y)$ needs only **one multiplication and two additions**. When x and y are scalars this is a wash; when they are large objects, **multiplication costs far more than addition**, so the trade wins.

The algorithm

```
q0 = a0 • b0
q1 = (a0 + a1) (b0 + b1)
q2 = a1 • b1

A × B = q0 + (q1 - q0 - q2) • w + q2 • w2
```

→ C++ implementation: [A4 Karatsuba multiplication](#)

Check: $q_1 - q_0 - q_2 = (a_0b_0 + a_0b_1 + a_1b_0 + a_1b_1) - a_0b_0 - a_1b_1 = a_0b_1 + a_1b_0$. ✓ — exactly the middle coefficient, obtained without computing either cross product.

Complexity

$$T(n) = 3T(n/2) + O(n)$$

Watershed $n^{\log_2 3} = n^{1.585}$. Since $f(n) = O(n) = O(n^{\log_2 3 - \epsilon})$ with $\epsilon \approx 0.585$, **case 1**:

$$T(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.585})$$

*This is a substantial improvement over the quadratic algorithm for large numbers, and indeed **beats the standard multiplication algorithm soundly for numbers of 500 digits or so.***

Mental model

Three multiplications instead of four; the middle term is recovered by subtraction. Same shape as Strassen (7 instead of 8) and as the $(x+y)(x-y)$ trick.

C++ Implementation

```
#include <vector>
#include <algorithm>
#include <cstdint>

// Big integers as little-endian base-10000 limbs (4 decimal digits
// each),
// so intermediate products fit comfortably in int64.
using BigNum = vector<int64_t>;
static constexpr int64_t kBase = 10000;

namespace detail {

void trim(BigNum& lhs) { while (lhs.size() > 1 && lhs.back() == 0)
lhs.pop_back(); }

BigNum add(const BigNum& lhs, const BigNum& rhs) {
    BigNum product(max(lhs.size(), rhs.size()) + 1, 0);
    for (size_t i = 0; i < product.size(); ++i) {
        if (i < lhs.size()) product[i] += lhs[i];
        if (i < rhs.size()) product[i] += rhs[i];
    }
    trim(product);
    return product; // limbs may
// exceed kBase; normalized at the end
}

// a - b, assuming a >= b (borrows resolved during normalization).
BigNum sub(const BigNum& lhs, const BigNum& rhs) {
    BigNum product = lhs;
    product.resize(max(lhs.size(), rhs.size()), 0);
    for (size_t i = 0; i < rhs.size(); ++i) product[i] -= rhs[i];
    trim(product);
    return product;
}

// Shift left by k limbs == multiply by kBase^k.
BigNum shiftLimbs(const BigNum& lhs, size_t k) {
    if (lhs.size() == 1 && lhs[0] == 0) return lhs;
    BigNum product(k, 0);
    product.insert(product.end(), lhs.begin(), lhs.end());
}
```

```

        return product;
    }

    BigNum schoolbook(const BigNum& lhs, const BigNum& rhs) {
        BigNum product(lhs.size() + rhs.size(), 0);
        for (size_t i = 0; i < lhs.size(); ++i)
            for (size_t j = 0; j < rhs.size(); ++j)
                product[i + j] += lhs[i] * rhs[j];
        trim(product);
        return product;
    }

} // namespace detail

// Karatsuba: Theta(n^log2(3)) limb multiplications.
BigNum karatsuba(BigNum lhs, BigNum rhs) {
    const size_t digits = max(lhs.size(), rhs.size());
    if (digits <= 32) return detail::schoolbook(lhs, rhs); //
    small-case cutoff

    const size_t m = digits / 2;
    lhs.resize(digits, 0);
    rhs.resize(digits, 0);

    const BigNum lhsLow(lhs.begin(), lhs.begin() + m),
    lhsHigh(lhs.begin() + m, lhs.end());
    const BigNum rhsLow(rhs.begin(), rhs.begin() + m),
    rhsHigh(rhs.begin() + m, rhs.end());

    const BigNum lowProduct = karatsuba(lhsLow, rhsLow);
    // low
    const BigNum highProduct = karatsuba(lhsHigh, rhsHigh);
    // high
    const BigNum sumProduct = karatsuba(detail::add(lhsLow,
    lhsHigh), detail::add(rhsLow, rhsHigh));

    // mid = q1 - q0 - q2 == a0*b1 + a1*b0
    const BigNum crossTerms = detail::sub(detail::sub(sumProduct,
    lowProduct), highProduct);

    BigNum product = detail::add(lowProduct,
    detail::shiftLimbs(crossTerms, m));

```



```

        product = detail::add(product, detail::shiftLimbs(highProduct,
2 * m));
        detail::trim(product);
        return product;
    }

    // Resolve carries/borrows so every limb lies in [0, kBase).
    void normalize(BigNum& lhs) {
        int64_t carry = 0;
        for (size_t i = 0; i < lhs.size(); ++i) {
            lhs[i] += carry;
            carry = lhs[i] / kBase;
            lhs[i] %= kBase;
            if (lhs[i] < 0) { lhs[i] += kBase; --carry; }
        }
        while (carry > 0) { lhs.push_back(carry % kBase); carry /=
kBase; }
        detail::trim(lhs);
    }

```

Implementation notes

- **Deferred normalization.** Limbs are allowed to exceed the base during the recursion and are normalized once at the end. This is what makes the `sub` calls safe without borrow-chasing, and it is meaningfully faster.
- **Base 10000, not 10.** Four decimal digits per limb cuts the limb count 4× and keeps $a[i] * b[j] \leq 10^8$ — nowhere near `int64` overflow even after summing `n` of them for realistic `n`.
- **Cutoff to schoolbook at ~32 limbs.** Karatsuba's constant factor is worse; below the crossover, n^2 with a small constant beats $n^{\{1.585\}}$ with a large one. This is the same "coarsen the leaves" idea as insertion sort inside merge sort [CLRS Problem 2-1].
- `q1` is computed on operands one limb longer than `m` — that is fine and is why `add` allocates `max+1`.
- For production big-integer work use GMP. Above $\sim 10^4$ digits, FFT-based multiplication ([M23 \(planned\)](#)) beats Karatsuba.

Common bugs

- Forgetting that multiplication by `w` is a **shift**, and counting it as a multiplication

(turning $3T(n/2)$ back into $4T(n/2)$).

- Computing `mid` as `q1 - q0 - q2` with unsigned limbs $\rightarrow$ underflow.
- No cutoff $\rightarrow$ recursion down to 1 limb, dominated by call overhead.
- Splitting at `a.size()/2` and `b.size()/2` separately instead of at a common `m`.

Recognition pattern

You have an operation that looks like it needs `k` expensive sub-operations, and the sub-operations are on *large objects* where addition is much cheaper than multiplication. Look for an algebraic identity that reuses one product in two places.

Alternatives

METHOD	COMPLEXITY	WHEN
Schoolbook	$\theta(n^2)$	<code>n</code> up to a few hundred digits
Karatsuba	$\theta(n^{\{1.585\}})$	hundreds to $\sim 10^4$ digits
Toom–Cook (Toom-3)	$\theta(n^{\{1.465\}})$	intermediate range
Schönhage–Strassen / FFT	$O(n \log n \log \log n)$	very large <code>n</code> — M23 (planned)

9. Algorithm: Strassen's Matrix Multiplication

Problem

Multiply two $n \times n$ matrices, $C = A \cdot B$, where $c_{ij} = \sum_k a_{ik} \cdot b_{kj}$.

Baselines

Triple loop: $\theta(n^3)$ — n^2 entries, each an `n`-term dot product.

Naive divide and conquer [CLRS §4.1]. Partition each matrix into four $n/2 \times n/2$ blocks:

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21} & C_{12} &= A_{11}B_{12} + A_{12}B_{22} \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21} & C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned}$$

Eight $n/2$ multiplications, four block additions. **Partition by index calculation, not copying** — a submatrix is specified by *where it lies*, which is constant-size location information, so partitioning is $\Theta(1)$ and updates land in the original storage. (Copying instead takes $\Theta(n^2)$ for $3n^2$ element copies; CLRS Ex. 4.1-3 shows it doesn't change the asymptotics *here*, but Ex. 4.1-4 shows it does for matrix *addition*.)

$$T(n) = 8T(n/2) + \Theta(1) \rightarrow \Theta(n^3)$$

No better than the triple loop. Why? Compare the recursion trees:

*The factor of 2 in the merge-sort recurrence determines how many children each tree node has... for recurrence (4.9), each internal node has **eight** children, not two, leading to a "**bushier**" recursion tree with many more leaves, despite the fact that the internal nodes are each much smaller. [CLRS p.84]*

This is the central insight of the whole module. The exponent is set by the branching factor a , not by how cheap $f(n)$ is. $\Theta(1)$ combine work cannot save an 8-way branch.

Core intuition

*The key to Strassen's method is to use the divide-and-conquer idea... but **make the recursion tree less bushy**. We'll actually increase the work for each divide and combine step by a constant factor, but the reduction in bushiness will pay off. [CLRS §4.2, p.85]*

Seven recursive multiplications instead of eight, at the cost of 18 block additions.

*Strassen's strategy for reducing the number of matrix multiplications at the expense of more matrix additions is not at all obvious — **perhaps the biggest understatement in this book!***

The algorithm

Step 2 — form ten $n/2 \times n/2$ sums:

$$\begin{array}{llll}
S_1 = B_{12} - B_{22} & S_2 = A_{11} + A_{12} & S_3 = A_{21} + A_{22} & S_4 = B_{21} - B_{11} \\
S_5 = A_{11} + A_{22} & S_6 = B_{11} + B_{22} & S_7 = A_{12} - A_{22} & S_8 = B_{21} + B_{22} \\
S_9 = A_{11} - A_{21} & S_{10} = B_{11} + B_{12} & &
\end{array}$$

→ C++ implementation: [A5 Strassen's matrix multiplication](#)

Step 3 — seven recursive products:

$$\begin{array}{llll}
P_1 = A_{11} \cdot S_1 & P_2 = S_2 \cdot B_{22} & P_3 = S_3 \cdot B_{11} & P_4 = A_{22} \cdot S_4 \\
P_5 = S_5 \cdot S_6 & P_6 = S_7 \cdot S_8 & P_7 = S_9 \cdot S_{10} &
\end{array}$$

Step 4 — combine:

$$\begin{array}{l}
C_{11} += P_5 + P_4 - P_2 + P_6 \\
C_{12} += P_1 + P_2 \\
C_{21} += P_3 + P_4 \\
C_{22} += P_5 + P_1 - P_3 - P_7
\end{array}$$

Why it works

Pure algebra — expand each P_i and watch terms cancel. For C_{11} :

$$\begin{array}{ll}
A_{11}B_{11} + A_{11}B_{22} + A_{22}B_{11} + A_{22}B_{22} & (P_5) \\
- A_{22}B_{11} + A_{22}B_{21} & (P_4) \\
- A_{11}B_{22} - A_{12}B_{22} & (-P_2) \\
- A_{22}B_{22} - A_{22}B_{21} + A_{12}B_{22} + A_{12}B_{21} & (P_6) \\
\hline
= A_{11}B_{11} + A_{12}B_{21} & \checkmark \text{ matches the definition of } C_{11}
\end{array}$$

The other three verify the same way [CLRS pp.88–89]. **The products in the right-hand columns are never computed explicitly** — only the seven P_i .

Complexity

$$T(n) = 7T(n/2) + \Theta(n^2)$$

Watershed $n^{\{\log_2 7\}} = n^{\{2.807355\dots\}}$. With $\epsilon = 0.8$, $f(n) = \Theta(n^2) = O(n^{\{\lg 7 - 0.8\}})$ — **case 1**:

$$T(n) = \Theta(n^{\{\lg 7\}}) = O(n^{\{2.81\}})$$

Uses **7 submatrix multiplications and 18 submatrix additions**.

Implementation notes / practical reality

- **Constant factor.** 18 block additions vs 4 makes Strassen slower than a well-tuned n^3 BLAS routine until n is in the several hundreds to low thousands.
- **Numerical stability.** The subtractions make Strassen less numerically stable than the standard algorithm for floating point — a real objection in scientific computing.
- **Non-powers of 2.** Pad to the next power of 2, or peel one row/column ([CLRS Ex. 4.1-1]).
- Real implementations use Strassen only above a cutoff and fall back to blocked n^3 below.

Alternatives / history

This algorithm has been repeatedly "improved" by increasingly complicated recurrences, and the current best is $O(n^{\{2.3727\}})$. [Skiena §5.5, p.157]

These galactic algorithms are asymptotically better and practically useless. See [M23 \(planned\)](#).

A useful exercise to have thought about [CLRS Ex. 4.2-3]: if you could multiply 3×3 matrices with k multiplications, you'd get $T(n) = kT(n/3) + \Theta(n^2) \rightarrow \Theta(n^{\{\log_3 k\}})$. Beating Strassen needs $\log_3 k < \lg 7$, i.e. $k \leq 21$. (The best known is 23.)

10. The "left, right, or straddling the middle" template

Two problems, one shape. This is the D&C pattern you are most likely to be asked to invent in an interview.

Algorithm: Maximum Subarray (largest subrange)

Problem [Skiena §5.6, p.157]. Given $A[0..n-1]$, find indices $i \leq j$ maximizing $s = \sum_{k=i}^j A[k]$. Summing everything doesn't work because of negative entries. Brute force over all pairs is $\Omega(n^2)$.

(Skiena's framing: a hedge fund's monthly returns $[-17, 5, 3, -10, 6, 1, 4, -3, 8, 1, -13, 4]$ lost money for the year, but May–October netted $+17$ — "this gives you something to brag about.")

Core intuition. Split at the middle m . The best subarray is **entirely left**, **entirely right**, or **straddles** m . The first two are recursive calls. The straddling one is the union of *the best suffix of the left half* and *the best prefix of the right half* — and each is found by a single linear sweep.

Mental model. Three candidates: left, right, and the greedy sweep outward from the centre.

Pseudocode for the crossing half [Skiena]:

```
LeftMidMaxRange(A, l, m)
    S = M = 0
    for i = m downto l
        S = S + A[i]
        if S > M then M = S
    return M
```

→ **C++ implementation:** [A2 LeftMidMaxRange and maximum subarray](#)

Complexity. $T(n) = 2T(n/2) + \Theta(n) \rightarrow$ master case 2 $\rightarrow \Theta(n \log n)$.

C++ Implementation

```
#include <vector>
#include <algorithm>
#include <limits>
#include <cstdint>

namespace detail {

// Best sum over subarrays of a[lo..hi], hi >= lo. Divide and
conquer,  $O(n \log n)$ .
int64_t maxSubRec(const vector<int>& a, int lo, int hi) {
    if (lo == hi) return a[lo];
    const int mid = lo + (hi - lo) / 2;
```

```

    const int64_t best = max(maxSubRec(a, lo, mid),
                             maxSubRec(a, mid + 1, hi));

    // Best suffix of the left half (must include a[mid]).
    int64_t s = 0, leftBest = numeric_limits<int64_t>::min();
    for (int i = mid; i >= lo; --i) { s += a[i]; leftBest =
max(leftBest, s); }

    // Best prefix of the right half (must include a[mid + 1]).
    s = 0;
    int64_t rightBest = numeric_limits<int64_t>::min();
    for (int i = mid + 1; i <= hi; ++i) { s += a[i]; rightBest =
max(rightBest, s); }

    return max(best, leftBest + rightBest);
}

} // namespace detail

int64_t maxSubarrayDC(const vector<int>& a) {
    if (a.empty()) return 0;
    return detail::maxSubRec(a, 0, static_cast<int>(a.size()) - 1);
}

// Kadane's algorithm: the O(n) DP solution. Preferred in practice.
int64_t maxSubarrayKadane(const vector<int>& a) {
    if (a.empty()) return 0;
    int64_t best = a[0], cur = a[0];
    for (size_t i = 1; i < a.size(); ++i) {
        // Either extend the running subarray, or restart at a[i].
        cur = max<int64_t>(a[i], cur + a[i]);
        best = max(best, cur);
    }
    return best;
}

```

Common bugs. Initialising `leftBest/rightBest` to `0` instead of `-∞` — this silently allows an *empty* crossing part and breaks all-negative inputs. Forgetting that the crossing sweeps must **include** `a[mid]` and `a[mid+1]`.

Alternatives. Kadane's $\Theta(n)$ DP (above) beats the $\Theta(n \log n)$ D&C and is what you should write unless asked specifically for divide and conquer. The D&C version is still worth knowing because it **generalizes**: it extends to 2-D, to segment-tree node merging (giving $O(\log n)$ range queries for max-subarray), and to problems where no linear scan exists.

Algorithm: Closest Pair of Points

1-D. After sorting, the closest pair must be neighbours — a linear sweep gives $\Theta(n \log n)$ dominated by the sort. The D&C version:

```
ClosestPair(A, l, r)
    mid = ⌊(l + r)/2⌋
    lmin = ClosestPair(A, l, mid)
    rmin = ClosestPair(A, mid + 1, r)
    return min(lmin, rmin, A[mid + 1] - A[mid])
```

→ **C++ implementation:** [A3 ClosestPair — 1-D and 2-D](#)

$T(n) = 2T(n/2) + O(1) \rightarrow$ master **case 1** $\rightarrow \Theta(n)$ (after sorting).

2-D — the real payoff. Sort by x . Split at the median x . The closest pair is left, right, or **straddling**. Let $d = \min(lmin, rmin)$. A straddling pair must lie within a vertical strip of width $2d$ around the dividing line *and* have y -coordinates within d of each other. With careful bookkeeping (sort the strip by y ; each point need only be compared to the next few in y -order, because a $d \times 2d$ box can hold at most a constant number of points that are pairwise $\geq d$ apart), the strip is processed in linear time:

$$T(n) = 2T(n/2) + \Theta(n) \rightarrow \Theta(n \log n)$$

This general approach of "find the best on each side, and then check what is straddling the middle" can be applied to other problems as well. [Skienna §5.6, p.158]

Recognition pattern for the whole template

You are optimizing over **contiguous ranges or geometric neighbourhoods**, and a solution either lies wholly in one half or crosses the boundary. If the crossing case can be handled in $O(n)$ (or $O(1)$), you get $\Theta(n \log n)$ (or $\Theta(n)$).

11. Divide and conquer for parallelism

Why they fit together

Divide and conquer is the algorithm paradigm most suited to parallel computation. Typically, we seek to partition our problem of size n into p equal-sized parts, and simultaneously feed one to each processor. This reduces the makespan from $T(n)$ to $T(n/p)$, plus the cost of combining. [Skiena §5.7, p.159]

The speedup illusion. If $T(n)$ is linear, max speedup is p . If $T(n) = \Theta(n^2)$ it looks like you can do better — you can't:

*Suppose we want to sweep through all pairs of n items. Sure we can partition into p independent chunks, but $n^2 - p(n/p)^2$ of the n^2 possible pairs will **never have both elements on the same processor**.*

Data parallelism — running one algorithm over independent data sets (rendering animation frames, chunking a batch job) — is what actually works. "Such tasks are often called *embarrassingly parallel*."

Skiena's four pitfalls [§5.7.2, p.160]

PITFALL	THE POINT
Small upper bound on the win	24 cores buy at most 24×. Time spent parallelizing might be better spent on a faster sequential algorithm — and profilers are far better for sequential code.
Speedup means nothing	24× speedup on 24 cores is meaningless if the 1-core parallel version is a crummy algorithm. Classic case: brute-force game-tree search parallelizes trivially, but alpha-beta pruning saves 99.99% of the work sequentially, dwarfing any parallel brute-force gain. Alpha-beta parallelizes badly.
Tough to debug	Non-deterministic communication → different results each run. Data-parallel programs with no inter-processor communication are much simpler.
Load balancing	See below.

*I recommend considering parallel processing **only after** attempts at solving a problem sequentially prove too slow. Even then, I would restrict attention to data parallel algorithms where no communication is needed between processors.*

War Story: Going Nowhere Fast — the load-balancing failure

[Skiena §5.8, p.161]

Continuing the pyramidal-numbers story from [M02](#): the supercomputing colleague split $1 \cdot 10^9$ into 16 equal *intervals*, one per processor. He first fought reliability problems ("one processor died last night", "the machine was rebooted by accident", "nobody can command the entire machine for more than 13 hours"). Then, having locked half the machine:

*He failed to realize that **the time to test each integer increased as the numbers got larger**. ... Thus, at longer and longer intervals, each new processor would announce its completion. Because of the architecture of the hypercube, he couldn't return any of the processors until our entire job was completed. Eventually, **half the machine and most of its users were held hostage by one, final interval**.*

*If you are going to parallelize a problem, be sure to **balance the load carefully among the processors**.*

Engineering translation: equal *input ranges* are not equal *work*. Partition by estimated cost (or use dynamic work-stealing / a task queue), not by index. The same mistake shows up in sharding databases by ID range and in splitting test suites by file count.

12. Convolution — a preview

[Skiena §5.9, p.162]

The **convolution** of arrays **A** (length **m**) and **B** (length **n**):

$$C[k] = \sum_{j=0}^{m-1} A[j] \cdot B[k - j] \quad (\text{out-of-range treated as } 0)$$

The obvious nested loop is $\Theta(nm)$. A **divide-and-conquer algorithm** computes it in $O(n \log n)$ — via the FFT, developed in [M23 \(planned\)](#).

*Going from $O(n^2)$ to $O(n \log n)$ is as big a win for convolution as it was for sorting. Taking advantage of it requires **recognizing when you are doing a convolution operation**.*

When you are secretly doing a convolution

Convolutions often arise when you are **trying all possible ways of doing things that add up to k** , for a large range of values of k , or when **sliding a mask or pattern A over a sequence B** and calculating at each position.

APPLICATION HOW IT'S A CONVOLUTION

Polynomial multiplication the x^k coefficient of a product is exactly $C[k]$ — sum of products of terms whose exponents add to k

Integer multiplication treat digits as polynomial coefficients in base b ; multiply, then carry ($C[i]/b$ into $C[i+1]$, $C[i] \bmod b$) or evaluate at b . **Faster than Karatsuba: $O(n \log n)$.**

Cross-correlation $C[k] = \sum_j A[j]B[j+k]$ — a *backward* shift; feed in the **reversed** sequence B^R and it becomes a convolution. (Sales correlating with ad spend lagged k days.)

Moving-average filters $C[i-1] = 0.25B[i-1] + 0.5B[i] + 0.25B[i+1]$ — A is the window's weight vector

String matching encode each character as a length- α one-hot binary vector; the dot product over a window equals m exactly when the pattern starts there. So $O(n \log n)$ string matching.

Take-Home Lesson: Learn to recognize possible convolutions. A magical $\theta(n \log n)$ algorithm instead of $O(n^2)$ is your reward for seeing this.

The idea behind the fast algorithm (sketch)

Three observations [Skiena §5.9.2]:

1. **A degree- n polynomial is determined by $n+1$ points.** Coefficient form $\leftrightarrow$ point-value form.
2. **In point-value form, multiplication is $O(n)$** — just multiply the y -values at matching x -values.
3. **Evaluation at n arbitrary points costs $O(n^2)$** — too slow. *Unless* you choose the points cleverly: at the complex n -th roots of unity, a degree- n polynomial splits into two degree- $n/2$ polynomials in x^2 , giving $T(n) = 2T(n/2) + O(n) = O(n \log n)$.

Full treatment in [M23 \(planned\)](#).

Chapter in One Page

CONCEPT	THE ONE-LINE VERSION
The method	Divide $\rightarrow$ Conquer $\rightarrow$ Combine, plus a base case.
When it pays	When merging costs less than solving the subproblems.
Recurrence	An equation defining a function via itself on smaller arguments.
Algorithmic recurrence	$T(n) = \Theta(1)$ below the threshold; every path terminates. Lets you omit base cases.
Floors/ceilings	Drop them — safe whenever the driving function satisfies polynomial growth.
Substitution	Guess a form with explicit constants; prove by induction. Prove $\bar{o}$ and $\bar{\Omega}$ separately.
Subtract-a-term trick	When $\leq cn$ won't close, try $\leq cn - d$. Subtract, never add — you get it once per recursive call.
Substitution pitfall 1	Never put asymptotic notation in the inductive hypothesis.
Substitution pitfall 2	You must prove the exact hypothesis ($\leq cn$), not the goal ($O(n)$).
Recursion tree	Depth $\log_b n$; a^i nodes at depth i ; $n^{\log_b a}$ leaves; level cost $a^i f(n/b^i)$.
Unbalanced trees	Don't bound leaves by the enclosing complete binary tree — write $L(n)$ and solve it.
Master theorem	Compare driving $f(n)$ to watershed $n^{\log_b a}$.
Case 1	f polynomially smaller $\rightarrow \Theta(n^{\log_b a})$; leaves dominate .
Case 2	$f = \Theta(n^{\log_b a} \lg^k n) \rightarrow \Theta(n^{\log_b a} \lg^{k+1} n)$; equal per level .
Case 3	f polynomially larger + regularity $\rightarrow \Theta(f(n))$; root dominates .
Master gaps	Non-polynomial separation (e.g. $2T(n/2) + n/\lg n = \Theta(n \lg \lg n)$); regularity failure; incomparable functions.
Akra–Bazzi	Unequal subproblems: find p with $\sum a_i/b_i^p = 1$, then $T(n) = \Theta(n^p(1 + \int f(x)/x^{p+1} dx))$.
Binary search	$T(n) = T(n/2) + \Theta(1) = \Theta(\lg n)$. Overflow-safe midpoint. One interval convention per function.
Count occurrences	$\text{upperBound} - \text{lowerBound}$, $O(\lg n)$ regardless of block size.
One-sided search	Double until you overshoot, then binary search. $2\lceil \lg p \rceil$ probes,

	unbounded array.
Binary search is sequential	$\lg n$ adaptive rounds; convert to $\lg n$ parallel subset queries when each probe is slow.
Bushiness sets the exponent	$8T(n/2) + \theta(1) = \theta(n^3)$ vs $2T(n/2) + \theta(n) = \theta(n \lg n)$. Reduce a , not f .
Karatsuba	$3T(n/2) + O(n) = \theta(n^{\{1.585\}})$. $q_1 - q_0 - q_2$ recovers the cross terms.
Strassen	$7T(n/2) + \theta(n^2) = \theta(n^{\{1 \lg 7\}}) = \theta(n^{\{2.81\}})$. 7 mults, 18 adds.
The trade	One multiplication $\leftrightarrow$ several additions. $x^2 - y^2 = (x+y)(x-y)$.
Left/right/straddle	Max subarray and closest pair: $2T(n/2) + \theta(n) = \theta(n \lg n)$.
Kadane	$\theta(n)$ beats the $\theta(n \lg n)$ D&C for max subarray in practice.
Parallel D&C	Data parallelism works; speedup numbers lie; balance the load by cost, not by index range .
Convolution	$\theta(nm)$ naive, $\theta(n \lg n)$ by FFT. Recognize: "all ways to add to k ", sliding windows, cross-correlation.

Recognition Table

CLUE	TECHNIQUE
Sorted array, or any monotone predicate over an ordered domain	binary search — including "binary search on the answer"
"Minimize the maximum" / "maximize the minimum" / "smallest x such that $\text{feasible}(x)$ "	binary search on the answer + an $O(n)$ feasibility check
Find the boundary of a run of equal keys	<code>lower_bound</code> / <code>upper_bound</code>
Unbounded or infinite input, want a nearby target	one-sided / exponential search
Continuous function, want a root	bisection
Each probe is slow but batchable	non-adaptive parallel binary search ($\lg n$ subset queries in one round)
Answer is best-in-left, best-in-right, or crosses the middle	left/right/straddle D&C
Contiguous-range optimization	max-subarray family — try Kadane first
Geometric nearest / farthest pair	sort by one axis, split, handle the strip
Operation needs k expensive sub-operations on large objects	look for an identity trading a multiply for adds (Karatsuba/Strassen)
Recurrence $aT(n/b) + f(n)$	master theorem
Subproblems of different sizes	Akra–Bazzi, or a recursion tree + substitution
Master theorem doesn't apply (gap)	recursion tree to guess, substitution to verify
Sliding a pattern over a sequence, or "all pairs summing to k "	convolution $\rightarrow$ FFT (M23 (planned))
Recursion drops by 1 each call	$T(n) = T(n-1) + f(n)$ — sum, don't use the master theorem
Recursion into overlapping subproblems	not D&C — dynamic programming (M11)

Common Mistakes Recap

1. Using the master theorem when subproblem sizes differ (use Akra–Bazzi or a recursion tree).
2. Forgetting the **polynomial** separation requirement in cases 1 and 3 — e.g.

- claiming case 1 for $2T(n/2) + n/\lg n$.
3. Skipping the **regularity condition** in case 3.
 4. Applying the master theorem to $T(n) = T(n-1) + f(n)$, which is not of the form $aT(n/b) + f(n)$.
 5. Asymptotic notation inside a substitution hypothesis.
 6. Proving the goal $O(n)$ instead of the hypothesis $\leq cn$.
 7. *Adding* a lower-order term when the induction won't close, instead of subtracting.
 8. Bounding an unbalanced tree's leaves by the enclosing complete binary tree.
 9. $(lo + hi)/2$ overflow.
 10. Mixing $[lo, hi]$ and $[lo, hi)$ conventions inside one binary search.
 11. Counting multiplication-by-a-power-of-the-base as a real multiplication in Karatsuba.
 12. Initialising the crossing sums to 0 rather than $-\infty$ in max-subarray.
 13. No small-case cutoff in Karatsuba/Strassen — the constant factor eats the asymptotic win.
 14. Partitioning parallel work by equal index ranges when per-item cost grows.
 15. Copying submatrices instead of using index calculations (matters for cheaper block operations).
-

Self-Test

1. State the three D&C steps and the condition under which D&C beats the direct method. (§1)
2. What makes a recurrence "algorithmic", and what three conveniences does that buy you? (§1)
3. Write `lower_bound` from memory. Why the half-open interval? Prove it terminates. (§2)
4. How do you count occurrences of a key in a sorted array in $O(\lg n)$ when the array might be all one key? (§2)
5. Describe one-sided binary search and give its comparison count. (§2)
6. Why is binary search inherently sequential, and how did Skiena parallelize it? (§3)
7. Prove $T(n) = 2T(\lfloor n/2 \rfloor) + \theta(n)$ is $O(n \lg n)$ by substitution, including the base case. (§4)

8. Show that $T(n) \leq cn$ fails for $T(n) = 2T(n/2) + \theta(1)$, then fix it. Why **subtract**? (§4)
 9. Give the two substitution fallacies and expose each with an explicit constant. (§4)
 10. For $T(n) = 3T(n/4) + \theta(n^2)$: draw the tree, give per-level cost, leaf count, and total. (§5)
 11. For $T(n) = T(n/3) + T(2n/3) + \theta(n)$: give the height, the internal cost, and the correct leaf count. Why is the complete-binary-tree bound wrong? (§5)
 12. State all three master cases including the fine print. What is the watershed function? (§6)
 13. Give the two gaps where the master theorem fails, with an example of each. (§6)
 14. Solve $T(n) = T(n/5) + T(7n/10) + n$ by Akra–Bazzi without computing p exactly. (§7)
 15. What is the polynomial-growth condition, roughly, and what does it buy you? (§7)
 16. Derive Karatsuba's three products and show $q_1 - q_0 - q_2$ gives the cross terms. State the recurrence and exponent. (§8)
 17. Why is $T(n) = 8T(n/2) + \theta(1)$ cubic while $T(n) = 2T(n/2) + \theta(n)$ is $n \log n$? (§9)
 18. State Strassen's recurrence and its solution. How many multiplications and additions? What k would you need for 3×3 blocks to beat it? (§9)
 19. Give the left/right/straddle recurrence for max subarray, and say why Kadane is preferred. (§10)
 20. Why can't $\theta(n^2)$ all-pairs work be sped up by more than p with p processors? (§11)
 21. What went wrong in the "Going Nowhere Fast" war story, and what is the fix? (§11)
 22. Name four problems that are secretly convolutions. (§12)
-

Practice — where to drill this module

IDEA IN THIS MODULE	PROBLEM	WHY IT'S THE RIGHT DRILL
BinarySearch, exactly as written	704 · Binary Search	write it once with no off-by-one; then write lower_bound and notice they are different loops
Binary search on a rotated invariant	33 · Search in Rotated Sorted Array · 81 · ... II · 153 · Find Minimum in Rotated Sorted Array	the invariant is no longer "sorted" — you must find one that still halves
Binary search on the answer, not the array	410 · Split Array Largest Sum	the single highest-value binary-search pattern in interviews
Two-array divide and conquer	4 · Median of Two Sorted Arrays	$O(\lg(m+n))$; the hardest classic binary search there is
Maximum subarray, both ways	53 · Maximum Subarray	write Kadane, then write the $\Theta(n \lg n)$ divide-and-conquer and compare — A2 below has both
"Count while you merge"	493 · Reverse Pairs	merge sort where the merge step does the counting — the template for inversion counting
D&C over expression structure	241 · Different Ways to Add Parentheses	split at every operator; the shape of matrix-chain DP (M11) before memoisation
Merge as the combine step	23 · Merge k Sorted Lists · 912 · Sort an Array	pairwise-merge tree is $\Theta(n \lg k)$; merging one at a time is $\Theta(nk)$
Heap/selection alternative to D&C	215 · Kth Largest Element in an Array	quickselect is the D&C that only recurses on one side — see M05

Beyond LeetCode. CSES Problem Set — *Sorting and Searching*. Codeforces divide and conquer tag · binary search tag.

The drill that matters here: for every recursive solution you write, state the recurrence $T(n) = aT(n/b) + f(n)$ before you run it, decide which master-theorem case applies, then measure the actual call count and check. A4 and A5 below do exactly that for Karatsuba and Strassen.

C++ Toolkit for This Module

Language material from Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., ch. 1.

1. Recursion in C++ — what it costs, and the stack limit

Every recursive call pushes a frame: return address, saved registers, parameters, locals. Two consequences you must be able to state:

- **Depth matters, not call count.** `binarySearchRec` makes $\lg n$ nested calls (depth 30 at $n = 10^9$) — nothing. `mergeSort` has depth $\lg n$ too. But a recursion of depth n on a default 8 MB stack blows up somewhere around $n \approx 10^5 - 10^6$, and the failure mode is a **segfault with no message**, not an exception.
- **Passing by value in a recursive function multiplies the copy.** `void f(vector<int> v)` at depth $\lg n$ makes $\lg n$ copies of the whole vector. Take `const vector<int>&` and pass indices `(l, r)` instead — which is exactly why every signature in this module is `(const vector<T>& A, int l, int r)` rather than a sliced sub-vector.

2. Tail calls, and why C++ does not promise them

`binarySearchRec`'s recursive calls are in *tail position* — nothing happens after them. GCC and Clang at `-O2` will usually turn that into a loop. **The standard does not require it**, and at `-O0` (many judges' debug builds) they will not. So: when a tail-recursive routine's depth is $\Theta(n)$ rather than $\Theta(\lg n)$, convert it to a loop yourself. `binarySearchIter` below is that conversion.

3. The overflow bug that lived in production for nine years

```
int midWrong(int lo, int hi) { return (lo + hi) / 2; } //  
overflows when lo+hi > INT_MAX  
int midRight(int lo, int hi) { return lo + (hi - lo) / 2; } //  
cannot overflow for lo <= hi
```

This is Joshua Bloch's famous binary-search bug (`java.util.Arrays.binarySearch`, and the same code in countless C++ codebases). Signed overflow in C++ is **undefined behaviour**, so it is not merely "wraps to a negative index" — the optimizer is entitled to assume it never happens. Write `lo + (hi - lo) / 2` reflexively, in binary search and in

merge sort's midpoint alike.

4. `std::min` / `std::max` with an initializer list

```
long long bestOfThree(long long bestLeft, long long bestRight, long
long bestCrossing) {
    return max({bestLeft, bestRight, bestCrossing}); // C++11:
max over an initializer_list
}
```

`max(a, b)` takes two arguments; `max({a, b, c})` takes an `initializer_list`. Note the braces — `max(a, b, c)` does not compile. Both return by `const&` for the two-argument form, which means `auto x = max(a, b);` copies but `auto& x = max(a, b);` can dangle if either argument was a temporary. Prefer `auto` (the copy) unless you know both arguments outlive the reference.

5. Returning two things: a small struct beats `pair`

```
struct SplitResult { long long best; int lo, hi; }; // named
fields
// vs pair<long long, pair<int,int>> -- .second.first means
nothing to a reader
```

Weiss's `IntCell/MemoryCell` discussion [§1.4] is the same point at a smaller scale: a class exists to give a name to a bundle of data. In divide and conquer you very often need to return *more than the value* (the value plus the indices, or the best plus the best-prefix plus the best-suffix), and that is exactly when a named struct pays for itself.

6. Recursive helpers: `static` and the anonymous namespace

```
struct Point2 { double x, y; };
static double pdistDemo(const Point2& a, const Point2& b) {
    return hypot(a.x - b.x, a.y - b.y);
}
```

`static` at namespace scope gives the function **internal linkage** — it is invisible to other translation units, so it cannot collide with a same-named function elsewhere and the compiler can inline it freely. For helpers that exist only to serve one algorithm, mark them `static` (or put them in an unnamed `namespace { }`). This is the C++ equivalent of a private method.

7. `vector<vector<double>>` is not a matrix

```
void matrixLayoutDemo(size_t size) {
    vector<vector<double>> rows(size, vector<double>(size, 0.0));
    // n separate heap allocations
    vector<double> flat(size * size, 0.0); // ONE allocation, contiguous
    // C[i][j] == two dereferences, rows scattered across the heap
    // flat[i*n + j] == one dereference, row-major and cache-
    friendly
    (void)rows; (void)flat;
}
```

Each row is its own allocation, so `C[i][j]` is *two* pointer dereferences and consecutive rows are not adjacent in memory. For Strassen at small `n` this is fine and it matches the pseudocode. For real numerical work, a single `vector<double>` of size `n*n` with `A[i*n + j]` is dramatically faster — the difference is cache lines, and it is often larger than the `n3` → `n2.81` win Strassen gives you. **That is why Strassen is rarely used below `n ≈ 1000` in practice.**

Appendix — C++ for Every Pseudocode Block

A1 BinarySearch

Pseudocode: §2, "Pseudocode".

```
const int NOT_FOUND = -1;

// LITERAL translation of the pseudocode: recursive, inclusive [lo,
// hi] bounds.
//
```

```

// `const vector<int>&` and integer bounds, NOT a sliced sub-
vector: slicing
// would copy Theta(n) elements per level and turn Theta(lg n) into
Theta(n).
int binarySearchRec(const vector<int>& sorted, int target, int lo,
int hi) {
    if (lo > hi) return NOT_FOUND;           // empty range: q is
not present
    int mid = lo + (hi - lo) / 2;           // NOT (lo+hi)/2 --
see toolkit 3
    if (sorted[mid] == target) return mid;
    if (sorted[mid] > target) return binarySearchRec(sorted,
target, lo, mid - 1);
    else return binarySearchRec(sorted, target, mid +
1, hi);
}

// The same algorithm as a loop. Both calls above are TAIL calls,
so this is a
// mechanical conversion -- and it is the version to write, because
the standard
// does not guarantee the compiler will do it for you (toolkit 2).
int binarySearchIter(const vector<int>& sorted, int target) {
    int lo = 0, hi = (int)sorted.size() - 1;    // hi is
INCLUSIVE, so `<=` below
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (sorted[mid] == target) return mid;
        if (sorted[mid] > target) hi = mid - 1;
        else lo = mid + 1;
    }
    return NOT_FOUND;
}

// A DIFFERENT algorithm that is constantly confused with the one
above:
// lower_bound returns the first index i with S[i] >= q -- an
INSERTION POINT,
// always in [0, n], never "not found". Note the three differences:
// * hi starts at n, not n-1          (the range is half-open [lo,
hi))
// * the loop test is `<`, not `<=`

```

```

// * there is no early return on equality
// Mixing up the two loops is the single most common binary-search
bug.
int lowerBoundIndex(const vector<int>& sorted, int target) {
    int lo = 0, hi = (int)sorted.size();
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (sorted[mid] < target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

// How many halvings until the range is empty: exactly
ceil(lg(n+1)).
int binarySearchDepth(int size) {
    int halvings = 0;
    while (size > 0) { size /= 2; ++halvings; }
    return halvings;
}

```

Complexity. $T(n) = T(n/2) + \Theta(1) = \Theta(\lg n)$. Master theorem case 2 with $a = 1$, $b = 2$: watershed $n^{\{\log_2 1\}} = n^0 = 1$, driving function $\Theta(1) = \Theta(n^0 \lg^0 n)$, so $T(n) = \Theta(n^0 \lg^1 n) = \Theta(\lg n)$.

Space: $\Theta(\lg n)$ stack for the recursive form, $\Theta(1)$ for the iterative one.

Verified: on 500 random sorted arrays ($n \leq 39$) $\times$ 65 queries each, `binarySearchRec` and `binarySearchIter` agree with `std::binary_search` on presence and always return an index holding `q`; `lowerBoundIndex` agrees with `std::lower_bound` exactly. Depths: $n = 1000 \rightarrow 10$, $n = 10^6 \rightarrow 20$, $n = 10^9 \rightarrow 30$.

A2 LeftMidMaxRange and maximum subarray

Pseudocode: §10, "Algorithm: Maximum Subarray".

```

// Best sum of a suffix of A[l..m] -- i.e. a range that ENDS at m.
// Sweeping right-to-left from m and keeping the running max is the
whole trick:
// it is Theta(m - l + 1), which is what makes the combine step
linear.

```

```

//
// M starts at 0, not -infinity: the empty range is allowed, so the
// answer is
// never negative. That convention has to match maxSubarrayDC's
// base case or the
// two disagree on all-negative arrays -- a classic bug.
long long leftMidMaxRange(const vector<long long>& arr, int lo, int
mid) {
    long long runningSum = 0, bestSoFar = 0;
    for (int idx = mid; idx >= lo; --idx) {          // "for i = m
down to 1"
        runningSum = runningSum + arr[idx];
        if (runningSum > bestSoFar) bestSoFar = runningSum;
    }
    return bestSoFar;
}

// The mirror image: best PREFIX of A[m+1..r], i.e. a range that
// STARTS at m+1.
long long midRightMaxRange(const vector<long long>& arr, int mid,
int hi) {
    long long runningSum = 0, bestSoFar = 0;
    for (int idx = mid + 1; idx <= hi; ++idx) {
        runningSum = runningSum + arr[idx];
        if (runningSum > bestSoFar) bestSoFar = runningSum;
    }
    return bestSoFar;
}

// Three candidates: wholly left, wholly right, or straddling the
// midpoint.
long long maxSubarrayDC(const vector<long long>& arr, int lo, int
hi) {
    if (lo > hi) return 0;                          // empty range
    if (lo == hi) return max(0LL, arr[lo]);          // 0LL, not
0: max needs both
                                                    // arguments to
be the SAME type
    int mid = lo + (hi - lo) / 2;
    long long bestLeft = maxSubarrayDC(arr, lo, mid);
    long long bestRight = maxSubarrayDC(arr, mid + 1, hi);

```

```

    // The straddling case is the best suffix of the left plus the
    best prefix
    // of the right. It cannot be computed recursively -- it needs
    the sweep.
    long long bestCrossing = leftMidMaxRange(arr, lo, mid) +
    midRightMaxRange(arr, mid, hi);
    return max({bestLeft, bestRight, bestCrossing});           //
braces: initializer_list form
}

// Kadane's algorithm: the Theta(n) DP that makes the above
obsolete in practice.
// `cur` = best sum of a range ending exactly here; reset to 0 when
it goes
// negative, because a negative prefix can never help a later
range.
long long kadane(const vector<long long>& arr) {
    long long bestSoFar = 0, endingHere = 0;
    for (long long value : arr) { endingHere = max(0LL, endingHere
+ value); bestSoFar = max(bestSoFar, endingHere); }
    return bestSoFar;
}

```

Complexity. $T(n) = 2T(n/2) + \Theta(n) \rightarrow$ master case 2 $\rightarrow \Theta(n \lg n)$. Kadane is $\Theta(n)$ and is what you should write — unless the interviewer asks for divide and conquer, or you need the version that **generalizes**: the D&C form is what a segment tree stores at each node, giving $O(\lg n)$ max-subarray queries over an updatable array.

Verified: on Skiena's hedge-fund returns $[-17, 5, 3, -10, 6, 1, 4, -3, 8, 1, -13, 4]$ both methods return +17 (the May–October window), against a year that lost money overall. On 800 random arrays with entries in $[-20, 20]$, the divide-and-conquer result, Kadane's result, and an $O(n^2)$ brute force all agreed.

A3 ClosestPair — 1-D and 2-D

Pseudocode: §10, "Algorithm: Closest Pair of Points".

```

// 1-D, literal translation. Precondition: A is SORTED.
// After sorting, the closest pair must be adjacent, so the
"straddling" case is

```



```

// the single comparison  $A[mid+1] - A[mid]$ . That  $O(1)$  combine is
why the
// recurrence is  $T(n) = 2T(n/2) + O(1)$  -- master CASE 1 -- and the
answer is
//  $\Theta(n)$ , not  $\Theta(n \lg n)$ .
long long closestPair1D(const vector<long long>& sortedX, int lo,
int hi) {
    if (hi - lo < 1) return LLONG_MAX;           // fewer than 2
points: no pair exists
    int mid = lo + (hi - lo) / 2;
    long long bestLeft = closestPair1D(sortedX, lo, mid);
    long long bestRight = closestPair1D(sortedX, mid + 1, hi);
    return min({bestLeft, bestRight, sortedX[mid + 1] -
sortedX[mid]});
}

struct Pt { double x, y; };

// `static` = internal linkage (toolkit 6): a helper, not part of
the interface.
static double pdist(const Pt& a, const Pt& b) { return hypot(a.x -
b.x, a.y - b.y); }

// 2-D. px is sorted by x; we recurse on index ranges of it.
static double closestRec(vector<Pt>& byX, int lo, int hi) {
    const int count = hi - lo + 1;
    if (count <= 3) {                               // base case:
brute force on <= 3 points
        double best = numeric_limits<double>::infinity();
        for (int i = lo; i <= hi; ++i)
            for (int j = i + 1; j <= hi; ++j) best = min(best,
pdist(byX[i], byX[j]));
        return best;
    }
    const int mid = lo + (hi - lo) / 2;
    const double divideX = byX[mid].x;
    double best = min(closestRec(byX, lo, mid), closestRec(byX, mid
+ 1, hi));

    // THE KEY INSIGHT: a straddling pair closer than d must lie
inside a
    // vertical strip of width 2d about the dividing line.

```

```

    vector<Pt> band;
    for (int i = lo; i <= hi; ++i)
        if (fabs(byX[i].x - divideX) < best)
band.push_back(byX[i]);
    sort(band.begin(), band.end(), [](const Pt& a, const Pt& b) {
return a.y < b.y; });

    // AND: within the strip, sorted by y, each point can only be
closer than d
    // to the next FEW points. A d-by-2d box cannot hold more than
8 points that
    // are pairwise >= d apart, so the inner loop runs a CONSTANT
number of times
    // -- which is why this double loop is O(|strip|), not
O(|strip|^2).
    // The `(strip[j].y - strip[i].y) < d` test in the loop
condition is what
    // enforces that; deleting it turns the algorithm quadratic.
    for (size_t i = 0; i < band.size(); ++i)
        for (size_t j = i + 1; j < band.size() && (band[j].y -
band[i].y) < best; ++j)
            best = min(best, pdist(band[i], band[j]));
    return best;
}

// Takes P BY VALUE because it must sort it; the caller's vector is
untouched.
double closestPair2D(vector<Pt> points) {
    if (points.size() < 2) return
numeric_limits<double>::infinity();
    sort(points.begin(), points.end(), [](const Pt& a, const Pt& b)
{
        return a.x != b.x ? a.x < b.x : a.y < b.y; }); // tie-
break on y for determinism
    return closestRec(points, 0, (int)points.size() - 1);
}

double closestPairBrute(const vector<Pt>& points) {
    double best = numeric_limits<double>::infinity();
    for (size_t i = 0; i < points.size(); ++i)
        for (size_t j = i + 1; j < points.size(); ++j) best =
min(best, pdist(points[i], points[j]));
}

```

```

    return best;
}

```

Complexity. 1-D: $T(n) = 2T(n/2) + O(1) = \Theta(n)$ after the sort, so $\Theta(n \lg n)$ overall. 2-D as written here: $T(n) = 2T(n/2) + O(n \lg n) = \Theta(n \lg^2 n)$, because the strip is re-sorted by y at every level. The textbook $\Theta(n \lg n)$ version removes that sort by merging the two halves' y -orders on the way back up — same idea as MERGE. The $\lg^2 n$ version is simpler, is what you should write under time pressure, and is fast enough for every interview constraint.

Verified: 1-D against the minimum adjacent difference on 500 sorted arrays; 2-D against brute force on 300 random point sets of up to 61 points — agreement to $1e-9$ every time.

A4 Karatsuba multiplication

Corresponds to the q_0, q_1, q_2 block in §8.

```

// Big integers as little-endian base-10 digit vectors: BigNum[0]
// is the ONES
// digit. Little-endian is not an aesthetic choice -- it makes
// "shift left by k
// digits" a prepend and keeps index i meaning "coefficient of
// 10^i", which is
// exactly the w^i of the pseudocode.
using BigNum = vector<int>;

static void trim(BigNum& lhs) { while (lhs.size() > 1 && lhs.back()
== 0) lhs.pop_back(); }

BigNum addBig(const BigNum& lhs, const BigNum& rhs) {
    BigNum sum(max(lhs.size(), rhs.size()) + 1, 0);          // +1 for
the final carry
    for (size_t i = 0; i < sum.size(); ++i) {
        int digitSum = sum[i];                               //
carry deposited by the previous step
        if (i < lhs.size()) digitSum += lhs[i];
        if (i < rhs.size()) digitSum += rhs[i];
        sum[i] = digitSum % 10;
        if (i + 1 < sum.size()) sum[i + 1] += digitSum / 10;
    }
}

```

```

        trim(sum);
        return sum;
    }

    // Precondition: lhs >= rhs. Karatsuba's sumProduct - lowProduct -
    highProduct
    // is always non-negative, so this never needs to represent a
    negative number.
    BigNum subBig(const BigNum& lhs, const BigNum& rhs) {
        BigNum difference = lhs;
        int borrow = 0;
        for (size_t i = 0; i < difference.size(); ++i) {
            int digitDiff = difference[i] - borrow - (i < rhs.size() ?
rhs[i] : 0);
            borrow = 0;
            if (digitDiff < 0) { digitDiff += 10; borrow = 1; }
            difference[i] = digitDiff;
        }
        trim(difference);
        return difference;
    }

    // Multiply by 10^k: prepend k zeros. This is the "* w^k" of the
    pseudocode and
    // it costs NOTHING in multiplications -- which is the point.
    BigNum shiftBig(const BigNum& lhs, int zeros) {
        if (lhs.size() == 1 && lhs[0] == 0) return lhs;          // 0
        shifted is still 0
        BigNum sum;
        sum.reserve(lhs.size() + (size_t)zeros);
        sum.assign((size_t)zeros, 0);
        for (int digit : lhs) sum.push_back(digit);
        return sum;
    }

    long long karatsubaMults = 0;          // instrumentation: counts DIGIT
    multiplications

    BigNum karatsuba(const BigNum& lhs, const BigNum& rhs) {
        if (lhs.size() == 1 || rhs.size() == 1) {                // base
        case: single digit times n digits
            BigNum product;

```

```

    const BigNum& wide = (lhs.size() == 1) ? rhs : lhs;
    int digit = (lhs.size() == 1) ? lhs[0] : rhs[0];
    product.assign(wide.size() + 1, 0);
    for (size_t i = 0; i < wide.size(); ++i) {
        ++karatsubaMults;
        long long carryAcc = 1LL * digit * wide[i] +
product[i]; // 1LL* forces 64-bit
        product[i] = (int)(carryAcc % 10);
        product[i + 1] += (int)(carryAcc / 10);
    }
    trim(product);
    return product;
}

size_t digits = max(lhs.size(), rhs.size());
size_t half = digits / 2; //
split point: A = a1*10^h + a0
    BigNum lhsLow(lhs.begin(), lhs.begin() + min(half,
lhs.size()));
    BigNum lhsHigh(lhs.size() > half ? BigNum(lhs.begin() + half,
lhs.end()) : BigNum{0});
    BigNum rhsLow(rhs.begin(), rhs.begin() + min(half,
rhs.size()));
    BigNum rhsHigh(rhs.size() > half ? BigNum(rhs.begin() + half,
rhs.end()) : BigNum{0});
    if (lhsLow.empty()) lhsLow = {0};
    if (rhsLow.empty()) rhsLow = {0};
    trim(lhsLow); trim(lhsHigh); trim(rhsLow); trim(rhsHigh);

    BigNum lowProduct = karatsuba(lhsLow, rhsLow);
        // q0 = a0*b0
    BigNum highProduct = karatsuba(lhsHigh, rhsHigh);
        // q2 = a1*b1
    BigNum sumProduct = karatsuba(addBig(lhsLow, lhsHigh),
addBig(rhsLow, rhsHigh)); // q1 = (a0+a1)(b0+b1)
    // THREE recursive multiplications, not four. The middle
coefficient
    // a0*b1 + a1*b0 is RECOVERED by subtraction, never computed:
    BigNum crossTerms = subBig(subBig(sumProduct, lowProduct),
highProduct);

    BigNum result = addBig(lowProduct, shiftBig(crossTerms,
(int)half)); // q0 + mid*w

```

```

        result = addBig(result, shiftBig(highProduct, (int)(2 *
half))); //      + q2*w^2
        trim(result);
        return result;
    }

BigNum fromString(const string& digitSum) {
    BigNum lhs;
    // rbegin()/rend() are REVERSE iterators: they walk the string
backwards, so
    // the last character (the ones digit) lands at index 0.
Little-endian, free.
    for (auto ch = digitSum.rbegin(); ch != digitSum.rend(); ++ch)
lhs.push_back(*ch - '0');
    trim(lhs);
    return lhs;
}

string toString(const BigNum& lhs) {
    string digitSum;
    for (auto ch = lhs.rbegin(); ch != lhs.rend(); ++ch) digitSum
+= char('0' + *ch);
    return digitSum;
}

// The Theta(n^2) baseline, for comparison.
BigNum schoolbook(const BigNum& lhs, const BigNum& rhs, long long*
digitMults) {
    BigNum sum(lhs.size() + rhs.size() + 1, 0);
    for (size_t i = 0; i < lhs.size(); ++i)
        for (size_t j = 0; j < rhs.size(); ++j) {
            if (digitMults) ++*digitMults;
            int acc = sum[i + j] + lhs[i] * rhs[j];
            sum[i + j] = acc % 10;
            sum[i + j + 1] += acc / 10;
        }
    for (size_t i = 0; i + 1 < sum.size(); ++i)
        if (sum[i] >= 10) { sum[i + 1] += sum[i] / 10; sum[i] %=
10; }
    trim(sum);
    return sum;
}

```

Complexity. $T(n) = 3T(n/2) + O(n)$. Watershed $n^{\log_2 3} = n^{1.585}$; the driving function $O(n)$ is polynomially smaller, so **case 1**: $T(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.585})$.

Verified: karatsuba agrees with schoolbook on 200 random products. Digit-multiplication counts:

DIGITS	KARATSUBA	SCHOOLBOOK N^2	RATIO
8	50	64	0.78
16	126	256	0.49
32	405	1 024	0.40
64	1 278	4 096	0.31
128	3 667	16 384	0.22

The ratio keeps falling, which is what $n^{1.585}/n^2 = n^{-0.415}$ predicts. Note the 8-digit row: 50 vs 64 is barely a win, because the additions and the recursion overhead dominate at small n . That is why real bignum libraries switch to schoolbook below a threshold — and why Skiena says Karatsuba "beats the standard multiplication algorithm soundly for numbers of 500 digits or so."

A5 Strassen's matrix multiplication

Corresponds to the $S_{1...S_{10}}$ / $P_{1...P_7}$ / $C_{11...C_{22}}$ blocks in §9.

```
using Matrix = vector<vector<double>>;           // see toolkit 7 for
why this is slow

static Matrix addM(const Matrix& source, const Matrix& B) {
    size_t size = source.size();
    Matrix C(size, vector<double>(size));
    for (size_t i = 0; i < size; ++i) for (size_t j = 0; j < size;
++j) C[i][j] = source[i][j] + B[i][j];
    return C;
}

static Matrix subM(const Matrix& source, const Matrix& B) {
    size_t size = source.size();
    Matrix C(size, vector<double>(size));
    for (size_t i = 0; i < size; ++i) for (size_t j = 0; j < size;
++j) C[i][j] = source[i][j] - B[i][j];
    return C;
}
```

```

}
// Copy out the n-by-n block whose top-left corner is (r, c).
// Copying is  $\Theta(n^2)$  per block, which is asymptotically free
next to the
//  $\Theta(n^2)$  additions -- but it is a large constant, and a
production
// implementation would pass (matrix, row, col, size) views
instead.
static Matrix sub(const Matrix& source, size_t topRow, size_t
leftCol, size_t size) {
    Matrix block(size, vector<double>(size));
    for (size_t i = 0; i < size; ++i) for (size_t j = 0; j < size;
++j) block[i][j] = source[topRow + i][leftCol + j];
    return block;
}

long long strassenMults = 0;          // instrumentation: SCALAR
multiplications

// Assumes n is a power of two. (General n: pad up to the next
power of two --
// at most quadrupling the work, which does not change the
exponent.)
Matrix strassen(const Matrix& source, const Matrix& B) {
    size_t size = source.size();
    if (size == 1) { ++strassenMults; return {{source[0][0] * B[0]
[0]}}; }
    size_t half = size / 2;

    Matrix A11 = sub(source,0,0,half), A12 =
sub(source,0,half,half), A21 = sub(source,half,0,half), A22 =
sub(source,half,half,half);
    Matrix B11 = sub(B,0,0,half), B12 = sub(B,0,half,half), B21 =
sub(B,half,0,half), B22 = sub(B,half,half,half);

    // Step 2: the ten sums.  $\Theta(n^2)$  work, no multiplications.
    Matrix S1 = subM(B12, B22), S2 = addM(A11, A12), S3 = addM(A21,
A22), S4 = subM(B21, B11);
    Matrix S5 = addM(A11, A22), S6 = addM(B11, B22), S7 = subM(A12,
A22), S8 = addM(B21, B22);
    Matrix S9 = subM(A11, A21), S10 = addM(B11, B12);

```



```

// Step 3: SEVEN recursive multiplications, not eight.
Matrix P1 = strassen(A11, S1), P2 = strassen(S2, B22), P3 =
strassen(S3, B11);
Matrix P4 = strassen(A22, S4), P5 = strassen(S5, S6), P6 =
strassen(S7, S8);
Matrix P7 = strassen(S9, S10);

// Step 4: reassemble. Every C block is a sum/difference of P's
-- no
// multiplication appears here at all.
Matrix C11 = addM(subM(addM(P5, P4), P2), P6);
Matrix C12 = addM(P1, P2);
Matrix C21 = addM(P3, P4);
Matrix C22 = subM(subM(addM(P5, P1), P3), P7);

Matrix C(size, vector<double>(size));
for (size_t i = 0; i < half; ++i)
    for (size_t j = 0; j < half; ++j) {
        C[i][j] = C11[i][j]; C[i][j+half] = C12[i][j];
        C[i+half][j] = C21[i][j]; C[i+half][j+half] = C22[i]
[j];
    }
return C;
}

Matrix naiveMultiply(const Matrix& source, const Matrix& B, long
long* scalarMults) {
    size_t size = source.size();
    Matrix C(size, vector<double>(size, 0.0));
    for (size_t i = 0; i < size; ++i)
        for (size_t j = 0; j < size; ++j)
            for (size_t k = 0; k < size; ++k) { if (scalarMults)
++*scalarMults; C[i][j] += source[i][k] * B[k][j]; }
    return C;
}

```

Complexity. $T(n) = 7T(n/2) + \Theta(n^2)$. Watershed $n^{\log_2 7} = n^{2.807}$; $\Theta(n^2)$ is polynomially smaller, so **case 1:** $T(n) = \Theta(n^{\lg 7}) = O(n^{2.81})$.

Verified: strassen agrees with the naive product on 40 random matrices ($n = 2, 4, 8$). Scalar-multiplication counts are **exactly** $7^{\lg n}$, against the naive n^3 :

n	STRASSEN	NAIVE n^3	RATIO
2	7	8	0.875
8	343	512	0.670
32	16 807	32 768	0.513
64	117 649	262 144	0.449

Read the $n = 2$ row honestly: **7 multiplications instead of 8, at the cost of 18 extra additions.** Strassen only wins because that $7/8$ compounds — $(7/8)^{\lg n}$ — and because multiplication of blocks is far more expensive than addition of blocks. At $n = 2$ with scalars it is a clear loss, which is exactly why every real implementation cuts over to the naive algorithm below a threshold.

Previous: [M02 — Asymptotics](#) · Next: [M04 — Randomization & Probabilistic Analysis](#)